Day 21 Task: File Permission Analysis

File Permission Analysis

File permission analysis is the process of examining file and directory permissions in a Linux system to identify unauthorized access and overly permissive configurations. Linux uses ownership and permissions to control who can read, modify, or execute files.

Incorrect permissions can expose sensitive information, allow unauthorized modifications, or enable attackers to misuse system resources. Analysing permissions helps administrators identify access-control weaknesses and improve system security.

Understanding Linux File Permissions

Linux assigns permissions to three categories of users: owner, group, and others.

Owner: The user who owns the file or directory.

Group: Users belonging to the group associated with the file or directory.

Others: All users who are neither the owner nor members of the associated group.

Linux provides three basic permission types:

- Read (r): Allows reading a file or listing directory entries.
- Write (w): Allows modifying a file or changing directory entries, subject to other restrictions.
- Execute (x): Allows executing a file or traversing a directory.

The ls -l command displays file permissions and ownership information.

ls -l

Example output:

-rwxr-xr-- 1 alice developers 1200 Oct 21 script.sh

Here, the owner has read, write, and execute permissions; the group has read and execute permissions; and others have read permission only.

Understanding Numeric Permissions

Linux permissions can also be represented using numeric values.

Permission	Value
Read (r)	4
Write (w)	2
Execute (x)	1

No permission 0

The values are added for each category to determine the permission number.

Permission 755

chmod 755 script.sh

The permission 755 means:

- Owner: 7 = read, write, execute (rwx)
- Group: 5 = read, execute (r-x)
- Others: 5 = read, execute (r-x)

This permission is commonly used for executable files and directories when that access is appropriate.

Permission 644

chmod 644 report.txt

The permission 644 means:

- Owner: 6 = read and write (rw-)
- Group: 4 = read only (r--)
- Others: 4 = read only (r--)

This is commonly suitable for files that the owner can edit and other users may read, provided the file does not contain sensitive information.

What Are Overly Permissive Configurations?

An overly permissive configuration grants more access than users or applications actually need. It may allow unauthorized users to view sensitive files, modify important data, or execute programs.

Examples include:

- Giving everyone write access to sensitive configuration files.
- Allowing unnecessary users to modify executable scripts.
- Making private files readable by all users.
- Granting excessive permissions to service accounts.
- Allowing unauthorized users to modify directories used by privileged applications.

Such configurations increase the attack surface and can make it easier for attackers to compromise a system.

Common Risky Permission Settings

Permission 777

chmod 777 file.txt

These grants read, write, and execute permissions to the owner, group, and others. It is usually inappropriate for sensitive files because any user with access to the file can modify it or execute it when applicable.

Permission 666

chmod 666 file.txt

This grants read and write permissions to everyone, but no execute permission. It can be dangerous for files that contain important data because other users may modify their contents.

World-Writable Directories

Directories that allow everyone to create, delete, or rename entries can create security risks if they are not appropriately controlled. Some world-writable directories, such as /tmp, are legitimate when configured with suitable protections, including the sticky bit.

How to Analyze File Permissions

File permission analysis can be performed using standard Linux commands.

Inspect permissions and ownership

ls -l

This displays permissions, owners, groups, file sizes, and modification times.

Inspect a specific file

stat report.txt

This displays detailed information about a file, including its permissions and ownership.

Find world-writable files

find /path/to/check -type f -perm -0002 -print

This searches the specified directory tree for files with the world-write permission bit set. Review the results to determine whether the permissions are justified.

Find world-writable directories

find /path/to/check -type d -perm -0002 -print

This identifies directories that allow others to write. Some results may be legitimate, so each one should be assessed in context.

Inspect file ownership

ls -l report.txt

Confirm that the owner and group are appropriate for the file's purpose.

These commands should be used on systems and directories you are authorized to inspect.

Correcting Insecure Permissions

When overly permissive settings are identified, permissions should be changed according to the file's intended use and the principle of least privilege.

For example, a private file can be restricted to its owner:

chmod 600 private.txt

This gives the owner read and write permissions while removing permissions from the group and others.

A general text document that should be readable by others but editable only by its owner can use:

chmod 644 report.txt

Ownership can be corrected when necessary:

sudo chown alice: developers report.txt

This changes the owner to alice and the associated group to developers.

Before changing permissions, verify the file's purpose and the access required by legitimate users and applications. Applying restrictive permissions without understanding the requirements can break services or prevent authorized access.

Security Best Practices

- Follow the principle of least privilege.
- Avoid using 777 or other excessively broad permissions without a justified need.
- Restrict write access to sensitive files and directories.
- Review ownership and permissions regularly.
- Protect configuration files containing credentials or other sensitive information.
- Use dedicated service accounts with only the required privileges.
- Investigate unexpected world-writable files and directories.
- Test permission changes before applying them to production systems.
- Maintain appropriate backups and document significant permission changes.

File permission analysis is an important part of Linux security. By examining ownership, reviewing numeric and symbolic permissions, and identifying overly permissive files and directories, administrators can reduce unauthorized access risks. Regular permission reviews and careful use of commands such as ls -l, stat, find, and chmod help maintain a more secure Linux environment.